


# NexusCore

A Unified, Database-Centric Project, Client and Bug Management System for HelaNexus Software Solutions

## Group - AC

- Target organization: HelaNexus Software Solutions (www.helanexus.com)
- Module lecturers: Mr. Saravanapavan Nasiketha, Mr. Savindu Dhamsara
- Module code: CS304.3 | Batch: 25.1/24.2

| Student name | Student ID | Role in the project |
|---|---|---|
| W.G.B.D. Gamage | 35273 | Backend, FastAPI, GitHub Integration |
| G.J.B.I. Perera | 36626 | Schema, Migrations, Data |
| R.G.M.N. Jayawardhana | 36909 | Stored Procedures, Transactions, Smart Assignment |
| N.H.M. Sivmini | 37097 | Triggers, Audit Chain, Activity Alerts |
| K.P.C.J. Bimsari | 37054 | Database Security (RBAC, RLS, MFA, Sessions) |
| H.K. Dinura | 37210 | Indexing, Search, Performance |
| H.I. Nirmal | 35172 | Analytics, Metrics, SQL Forecasting |
| M.F.S. Zainab | 36484 | Database Architect, Documentation,Backend |
| B.S.S.T. Cooray | 37226 | Frontend, UI/UX, Sound FX, Desktop Agent, QA, Gemini AI Agent, Onboarding Guide |
| H.C.S. Dilruk | 35408 | DevOps, Cloud, Backup/Recovery |

# 1. Project Title

NexusCore: A Unified, Database-Centric Project, Client and Bug Management System for HelaNexus Software Solutions

# 2. Introduction

Small software companies often build their internal tools one piece at a time. Proposals live in spreadsheets, sprints on a task board, client records in a separate admin portal, and bug reports in chat messages. Over time the same information is spread across many places. The result is duplicate entries, project statuses that do not match, unbilled hours, and many engineering hours spent on manual coordination.

This project tackles that problem inside HelaNexus Software Solutions. We propose NexusCore, a web application built on a PostgreSQL database. It is more than a simple CRUD system. Business rules, audit trails, security policies and reporting logic are placed inside the database itself. On top of this, NexusCore adds a modern dark interface with smooth animations, sound feedback, a Google Gemini AI assistant, and an interactive onboarding guide. Projects, GitHub bugs and developer work sessions come together in one system.

# 3. Organization Overview

HelaNexus Software Solutions is an active software company that builds custom web and mobile applications, SaaS products, workflow automation and IT consulting. A small agile team of 2 to 5 people (engineers, UI/UX designers, DevOps staff and trainees) runs 3 to 5 client projects at the same time, along with in-house products and maintenance contracts. Its work covers four areas: engineering, UI/UX design, QA/testing and administration.

The company currently keeps its information in four separate places: an internal Admin Panel (client onboarding and billing logs), GitHub and GitHub Projects (code, issues, sprint boards), Google Docs and Sheets (requirements, proposals, meeting minutes), and email and chat (client communication and status updates).

# 4. Problem Identification

## 4.1 How we investigated

We interviewed the Founder and CEO of HelaNexus, Kavindu Kaveesha, on 27/10/2026 using a 24-question protocol. It covered workflows, data bottlenecks, security expectations, database requirements and target metrics. The CEO gave formal permission to use HelaNexus as our case study

## 4.2 The core problem

HelaNexus has no single database that links clients, projects, staff, tasks and bugs. Information is copied between tools by hand, which causes these problems:

- **Scattered data: Information is spread across disconnected tools and synced manually (Q9).**
- **Duplicates and mismatches: Duplicate client profiles and inconsistent task statuses lead to invoice errors (Q10).**
- **Slow retrieval: Finding old project and task records wastes billable engineering time (Q11).**
- **Out-of-sync systems: The Admin Panel and GitHub issues show different statuses, which confuses sprint planning (Q12).**
- **Manual reporting: Preparing the weekly client progress report takes several hours of senior engineers' time (Q13).**
- **No audit trail: Nothing records who changed what, so scope creep and accidental overwrites go unnoticed (Q17).**

### 4.3 Additional requirements

After lecturer review, two requirements were added. First, GitHub bugs must enter through a separate staging layer, so external webhooks never write directly to the main database. Second, the system must detect non-work application use during logged work sessions and alert the project manager. We also added four enhancements: an animated dark interface, sound effects with mute controls, a Gemini AI agent for natural-language queries and sprint summaries, and a role-based onboarding guide.

## 5. Proposed Solution

NexusCore is a web platform powered by PostgreSQL 16+. It stores clients, projects, employees, tasks, GitHub bugs, work sessions and audit history in one relational schema. Its guiding idea is that data rules, auditability and security are enforced by the database, not left to the frontend code. The solution has four parts.

Reliable, automated data. Constraints and foreign keys prevent duplicate clients and invalid task states. Stored procedures automate weekly progress roll-ups, workload checks and duplicate client merging. Materialized views pre-compute cycle time, bug resolution speed, workload and project risk, so executive dashboards load instantly.

Security and accountability. Database triggers record every INSERT, UPDATE and DELETE in a hash-chained audit table, so any tampering is detectable. PostgreSQL roles, Row-Level Security, Argon2 password hashing and TOTP multi-factor login protect financial and client data from people who should not see it. Docker deployment with WAL archiving, off-site backups and Point-in-Time Recovery protects against data loss.

Integrations and smart features. A separate Python Sync Service pulls GitHub issues into a staging table, and a database procedure validates them and inserts clean records in a single transaction. A lightweight desktop agent reports the active window title during work sessions, and triggers raise a manager alert when non-work use passes a threshold. A scoring function recommends the best developer for each task based on skills, free capacity and deadline. The Gemini AI agent answers questions in natural language (Text-to-SQL), writes sprint summaries and helps triage bugs, and every AI interaction is logged.

User experience. The interface uses Tailwind CSS, shadcn/ui and Framer Motion for smooth transitions, animated charts, a project dependency graph and milestone celebrations. A Web Audio sound engine gives audio feedback with a mute and volume control in the header. Driver.js tours, tailored to each role (CEO, Manager, Developer, Client), and a "Getting Started" checklist help new users learn the system quickly.

## 6. Project Aim and Objectives

Aim: To design, build and evaluate a secure, database-centric project, client and bug management platform for HelaNexus that removes data fragmentation and applies advanced database concepts, supported by AI assistance, a polished interface and guided onboarding, within a 10-week development cycle.

### ❖ Specific objectives

1. Design an EER model and a normalized schema (3NF/BCNF) for 27 entities by Week 2. Measure: the schema passes constraint and foreign key tests.
2. Build at least 6 stored procedures (assignment, weekly roll-ups, staging sync) by Week 5. Measure: all covered by pgTAP tests.
3. Build at least 6 triggers (hash-chained audit, status history, activity alerts) by Week 6. Measure: tampered audit hashes are detected and rejected.
4. Implement ACID transactions with row-level locking (SELECT FOR UPDATE) for task allocation by Week 4. Measure: parallel load tests show no double allocation.
5. Enforce RBAC, Row-Level Security, TOTP MFA and revocable sessions by Week 3. Measure: restricted roles cannot see billing or audit data.
6. Apply B-tree, GIN, Trigram and BRIN indexing by Week 6. Measure: at least 50% lower query time, shown with EXPLAIN (ANALYZE, BUFFERS).
7. Develop the database-driven smart assignment and materialized-view dashboards by Week 7. Measure: the best developer appears in the top 3 in at least 75% of tests, and the weekly report builds in under 2 seconds instead of 3 to 4 hours.
8. Build the decoupled GitHub Sync Service with staging tables by Week 5. Measure: issues are parsed and upserted correctly and duplicate payloads are rejected.
9. Build the desktop activity agent and trigger-based manager alerts by Week 6. Measure: exceeding the threshold sends a real-time WebSocket alert.
10. Deploy to a cloud VM with WAL archiving and Point-in-Time Recovery by Week 8. Measure: a restore drill to a chosen timestamp succeeds without data loss.
11. Integrate the Gemini AI agent by Week 8. Measure: safe natural-language queries, sprint summary in under 5 seconds, every interaction logged.
12. Add automated testing with CI/CD, the animated interface, sound effects and onboarding guide by Weeks 9 and 10. Measure: GitHub Actions runs pgTAP, pytest, Vitest and Playwright on every pull request, and all evaluated users complete the onboarding tour.

## 7. Scope of the Project

### 7.1 Target users

- **Administrator / CEO: full access to finance, billing, projects, employees, audit logs, AI executive reports and monitoring rules.**
- **Project Manager: project lifecycle, sprint planning, smart assignment, GitHub sync review, live activity alerts, sprint summaries and report exports.**
- **Developer / QA Engineer: assigned tasks, bug tracking, time logging, work sessions, personal activity history, AI bug triage and onboarding.**
- **Client (restricted): read-only view of milestones, releases, burndown charts and approved progress summaries.**

### 7.2 Main features by priority

- **Must have: the 27-entity relational schema with constraints, RBAC, Row-Level Security, audit triggers, stored procedures, transactions, GitHub sync with staging, backup and PITR, and the CI/CD pipeline.**
- **Should have: executive dashboards, materialized views, smart assignment, the desktop activity agent, the Gemini AI agent, animations, sound effects, the onboarding guide and TOTP MFA.**
- **Stretch goals: Gemini parsing of client contract PDFs, audit log partitioning and 3D graphics on the login screen.**

### 7.3 System boundaries and assumptions

- **NexusCore does not replace GitHub for code hosting or reviews. It only reads issues through the REST API.**
- **Development uses synthetic data. The desktop agent records active process names only, with no keystrokes or screenshots, in line with Sri Lanka's Personal Data Protection Act No. 9 of 2022.**
- **Gemini works only through read-only, parameterized database connections. Sensitive financial columns are hidden from prompts and all AI activity is audited.**
- **Sound effects follow WCAG 2.1: each one has a visual notification and can be muted.**
- **Work is organized into five 2-week sprints, and the database schema is frozen at the end of Week 2.**

## 8. Advanced Database Concepts

Each concept below was chosen because it solves a specific problem found in the HelaNexus interview.

- Advanced design and normalization: EER modeling, 3NF/BCNF, keys, CHECK constraints and junction tables stop duplicate clients and invalid task states (Q10, Q16).

- Stored procedures and functions: Task assignment, weekly reports, client merging and GitHub staging ingestion run inside the database, which removes the manual reporting effort (Q13, Q20).
- Triggers: BEFORE/AFTER triggers write audit records (JSONB old and new values), keep status history, check workloads and raise alerts, without depending on application code (Q17).
- ACID transactions and concurrency: Transactions with SELECT FOR UPDATE row locks prevent two managers from allocating the same developer at once.
- Indexing and query optimization: B-tree indexes on foreign keys, partial indexes on active tasks, GIN on JSONB, trigram indexes for fuzzy client search and BRIN on logs keep searches at least 50% faster as data grows (Q11, Q20).
- Views and Row-Level Security: Restricted views and RLS policies keep client financial records away from developers while giving managers scoped access (Q18).
- Cryptographic security: Argon2 passwords, pgcrypto column encryption and revocable session tokens protect data even if a database dump leaks (Q18, Q21).
- Backup and recovery: WAL archiving with pgBackRest or WAL-G and regular restore drills protect against server failure and accidental deletion (Q19).
- Materialized views and analytics: Scheduled concurrent refreshes power delivery metrics and risk scores without slowing the live tables.
- In-database ranking and partitioning: A SQL scoring function gives transparent assignment recommendations, and monthly partitioning of the ACTIVITY_EVENT table keeps queries fast and old data easy to remove.
- AI audit logging: The AI_INTERACTION_LOG table records prompts, generated SQL, latency and token use for every Gemini request.

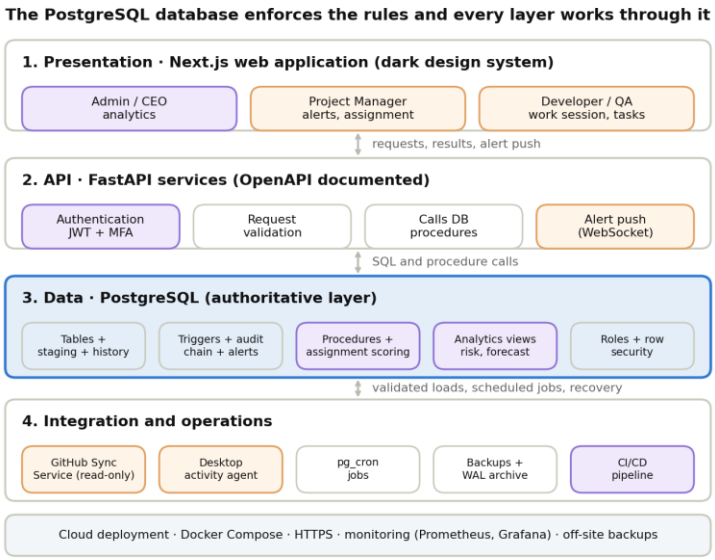
## 9. Proposed Technology Stack

- Database: PostgreSQL 16+ with pg_trgm, pgcrypto, pg_cron and pg_stat_statements. It supports RLS, triggers, ACID transactions, materialized views and WAL-based recovery.
- Backend: Python 3.12 with FastAPI, SQLAlchemy, Pydantic and psycopg3, chosen for fast async performance, automatic API documentation and strict validation.
- Frontend: Next.js 14, TypeScript, Tailwind CSS and shadcn/ui for a fast, modular, responsive dark interface, with Framer Motion, Lucide Icons and Canvas Confetti for animation.
- Audio and onboarding: Web Audio API / Howler.js for sound effects, and Driver.js / React Joyride for guided tours.
- AI: Google Gemini API (Gemini 1.5 Flash for fast answers, Pro for deeper analysis) for Text-to-SQL, summaries and bug triage.
- Security: JWT with hashed refresh tokens, Argon2 and pyotp (TOTP), mapped to PostgreSQL database roles.

- Integration and migrations: a Python Sync Service using the GitHub REST API, and Alembic for versioned schema migrations.
- DevOps and testing: Docker, Docker Compose, GitHub Actions, a Linux cloud VM, Prometheus and Grafana, with pgTAP (database), pytest (API), Playwright (end-to-end) and k6 (load) tests.

## 10. Preliminary System Architecture

NexusCore has four layers, with PostgreSQL as the single source of truth (Figure 1). The Next.js frontend sends authenticated requests to FastAPI. FastAPI checks the session, calls parameterized stored procedures and pushes real-time updates over WebSocket. The Gemini agent only reads from restricted views, and the GitHub Sync Service writes issues into a staging queue instead of the main tables.




## 11. Database Design Approach

### 11.1 Initial entities

The design has 27 entities in six groups:

- **Core business (11): CLIENT, CLIENT_INTERACTION, PROJECT, EMPLOYEE, APP_USER, PROJECT_ASSIGNMENT, TASK, BUG, TIME_LOG, GITHUB_REFERENCE, AUDIT_LOG**
- **GitHub integration (2): SYNC_RUN, GITHUB_STAGING**

- **Work monitoring (4): WORK_SESSION, ACTIVITY_EVENT (partitioned), APP_RULE, ALERT**
- **Intelligence and analytics (7): SKILL, EMPLOYEE_SKILL, TASK_SKILL, TASK_DEPENDENCY, TASK_STATUS_HISTORY, PROJECT_METRIC_SNAPSHOT, ASSIGNMENT_SUGGESTION**
- **Security and sessions (2): USER_SESSION, LOGIN_ATTEMPT**
- **AI (1): AI_INTERACTION_LOG**

### 11.2 Key relationships

A client owns many projects, and foreign keys prevent orphaned client deletions. Employees and projects have a many-to-many relationship resolved by PROJECT_ASSIGNMENT, and each task and bug is linked to one assigned developer. SYNC_RUN tracks every GitHub fetch, and raw payloads in GITHUB_STAGING are validated before they are upserted into BUG. WORK_SESSION owns many ACTIVITY_EVENT rows, and a match with a non-work APP_RULE creates an ALERT. EMPLOYEE_SKILL and TASK_SKILL feed the suggest_assignee function, and AI_INTERACTION_LOG links every AI request to its APP_USER.

### 11.3 Design plan

1. **Conceptual model: draw the extended EER diagram with all rules and cardinalities.**
2. **Logical schema: map it to relational tables in 3NF/BCNF and document candidate keys.**
3. **Physical design: add CHECK constraints, ENUM types, indexes and range partitioning.**
4. **Migrations and data: write versioned Alembic scripts and generate 10,000+ synthetic records for benchmarking.**
5. **Verification: review the schema with HelaNexus technical leaders before frontend integration.**

## 12. Development Methodology

The team will use Agile Scrum with five 2-week sprints, daily standups, a GitHub Projects Kanban board and automated CI/CD checks. Every sprint ends with a stakeholder review. The main implementation stages are database design, core database logic, integrations and monitoring, analytics and AI, and finally testing, evaluation and delivery.

The main risks and how we handle them:

- **Large scope: work is split among 10 members with clear subsystem owners, and the schema is frozen at Week 2.**
- **Little real data: synthetic data generators create realistic client portfolios and 10,000+ task records for performance tests.**

- **Monitoring privacy: monitoring runs only during work sessions, with no keylogging, and the rules are reviewed with HelaNexus for PDPA compliance.**
- **AI mistakes: Gemini uses read-only credentials, and generated queries are checked against allowed schemas before they run.**

## 13. Expected Outcomes

### 13.1 Deliverables

- **A working web application: Next.js frontend, FastAPI backend and PostgreSQL 16+ database**
- **The animated dark interface with sound effects, the Gemini AI agent and the role-based onboarding guide**
- **The GitHub bug synchronizer and the desktop activity agent**
- **Database documentation: EER diagram, normalized schema, migration scripts, pgTAP test suites and PITR drill logs**
- **Test results, performance benchmarks, usability evaluation results and user manuals**
- **A final demonstration to HelaNexus with documented feedback**

### 13.2 Benefits for HelaNexus

- **One source of truth for clients, projects, tasks and bugs**
- **No manual reporting: automated weekly reports and sprint summaries return 3 to 4 hours per manager each week**
- **No duplicate records, thanks to UNIQUE constraints and fuzzy matching**
- **Automatic GitHub bug capture with no double entry**
- **Fairer workloads, through data-driven assignment suggestions**
- **Faster adoption, through guided onboarding and clear feedback**

## 14. Project Timeline

The 10-week plan follows five sprints, each ending in a milestone.

- **Sprint 1 (Weeks 1 to 2): Finalize requirements, EER design, Docker and CI/CD setup. Build the 3NF/BCNF schema, migrations and synthetic data. Milestone M1: schema frozen and approved.**
- **Sprint 2 (Weeks 3 to 4): RBAC, Row-Level Security, TOTP MFA and sessions. Stored procedures, ACID transactions and row locking. Client, project, task and bug workflows. Milestone M2: core database system working.**

- **Sprint 3 (Weeks 5 to 6): Audit triggers, GitHub Sync Service, indexing benchmarks, partitioning, desktop activity agent and alert triggers. Milestone M3: integrations and telemetry complete.**
- **Sprint 4 (Weeks 7 to 8): Materialized view dashboards, delivery metrics, smart assignment, sound engine, Gemini AI agent, cloud deployment and PITR restore drill. Milestone M4: AI and cloud ready.**
- **Sprint 5 (Weeks 9 to 10): Onboarding guide, end-to-end tests, animation and accessibility polish, usability evaluation with HelaNexus staff, final demonstration to the CEO and final report. Milestone M5: final delivery.**

## 15. References


1. **R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.**
2. **A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.**
3. **T. Connolly and C. Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th ed. Pearson, 2015.**
4. **PostgreSQL Global Development Group, PostgreSQL Documentation. https://www.postgresql.org/docs/**
5. **Google, Google Gemini API Documentation. https://ai.google.dev/docs**
6. **GitHub, GitHub REST API and Webhooks Documentation. https://docs.github.com/en/rest**
7. **Parliament of Sri Lanka, Personal Data Protection Act, No. 9 of 2022.**
8. **D. M'Raihi, et al., TOTP: Time-Based One-Time Password Algorithm, RFC 6238, IETF, 2011.**
9. **OWASP Foundation, Application Security Verification Standard (ASVS), version 4.0.3**.

## Appendices

Interview Link: - HelaNexus_ADBMS_Interview_Report (1).docx

